

LECTURE 11

Reinforcement Learning Part 2: Policy Gradient Methods

REINFORCE • baselines • actor–critic • A2C/A3C • PPO • continuous control • SAC
Case study: 4-drone autonomous swarm, continuous waypoint control, 6-DoF flight dynamics, PPO vs SAC

AK DHAMIJA, Researcher & Instructor

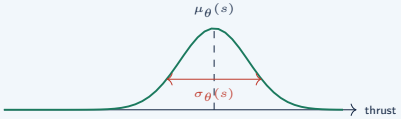
AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

When DQN Is Not Enough — Q-Learning Needs a Discrete Action Space

Value-based (DQN) limitations

Policy $\pi(s) = \arg\max_a Q(s, a; \theta)$ works when the action space is DISCRETE and SMALL ($|A| = 7$ in the wargame). It breaks down for: **continuous** actions — drone thrust $\in [-100, +100]^3$ N has infinitely many values, so the argmax becomes an optimisation inside every step; **huge discrete** spaces — 50,000-token vocabularies mean 50K Q-values per pass; and exploration is bolted on (ε-greedy).



the policy outputs a distribution; sample, don't argmax

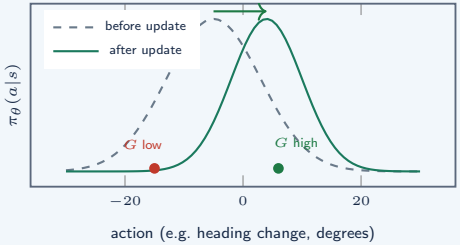
POLICY GRADIENT THEOREM

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t\Big]$$

$$G_t = \sum_{k \geq t} \gamma^{k-t} r_{k+1}$$

- Parameterise the POLICY $\pi_\theta(a|s)$ directly instead of learning values as an intermediary
- "Make actions that led to high return MORE probable by moving θ along the gradient of their log-probability"
- Continuous actions become trivial: $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2)$
- A stochastic policy gives **natural exploration**

Policy Gradient Intuition — Shifting Probability Toward Good Actions



Why the log-probability?

$\nabla_\theta \log \pi_\theta(a|s) = \nabla_\theta \pi_\theta(a|s)/\pi_\theta(a|s)$ — the **score function**. It tells us how to change θ to make action a more likely; dividing by π avoids over-reinforcing actions that are already frequent.

Analogy: after-action reviews. Tactics used in successful sorties get trained more often; tactics used in failed ones are de-emphasised — without anyone writing down the "correct" manoeuvre for each situation.

But a whole sortie's outcome is a noisy verdict on any single decision — the variance problem (three slides ahead).

REINFORCE (Williams 1992) — Complete Episodes, Then Update

Algorithm

Initialise θ randomly. For each episode: **1.** run π_θ to generate $\tau = (s_0, a_0, r_1, \ldots, s_T)$; **2.** for each t compute $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_{k+1}$; **3.** update $\theta \leftarrow \theta + \alpha\, G_t \nabla_\theta \log \pi_\theta(a_t|s_t)$. Batch form: $\nabla J \approx \frac{1}{T} \sum_t G_t \nabla_\theta \log \pi_\theta(a_t|s_t)$.

REINFORCE UPDATE

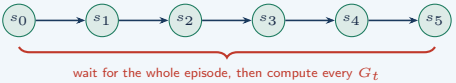
$$\theta \leftarrow \theta + \alpha\, G_t\, \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Strengths

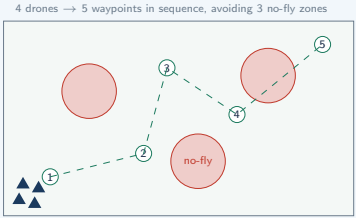
✓ unbiased gradient estimate (Monte Carlo) ✓ works for discrete AND continuous actions ✓ simple to implement.

Weaknesses

× HIGH VARIANCE: G_t sums a whole noisy trajectory × needs complete episodes before any update × sample-inefficient: each episode used once × can forget early experience.



Drone Swarm Case Study — Setup and the REINFORCE Baseline

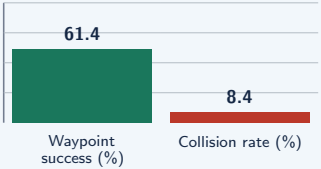


State and action

Per drone 18 features (position, velocity, fuel, 8 obstacle clearances, waypoint relative position) $\times$ 4 drones $=$ 72 $+$ swarm centroid (3) $+$ formation metrics (5) $=$ **80**. Action: continuous thrust $[F_x, F_y, F_z]$ per drone $=$ **12-dim**.

REINFORCE (no baseline)

Network $80 \to 256 \to 256 \to \mu(12) + \log \sigma(12)$; 10,000 episodes, batch 1.



Average return $142.8 \pm \mathbf{84.6}$: very high variance $\Rightarrow$ unstable learning. 6,200 episodes to reach 50% success.

The Variance Problem — A High Return Does Not Mean the Action Was Good

Episode 1: drone A makes a turn 5° off optimal at $t = 3$; the episode goes well for other reasons ($G_t = 180$). $\Rightarrow$ the update **increases** the probability of the poor turn.

Episode 2: the same drone makes a good turn; bad fuel luck ($G_t = 40$). $\Rightarrow$ the update **decreases** the probability of the good turn.

Root cause: G_t mixes the quality of the specific action a_t with all the luck and skill of the rest of the episode. We want credit only for the *relative* value of a_t against the alternatives.

BASELINE: UNBIASED VARIANCE REDUCTION

$$\theta \leftarrow \theta + \alpha \left(G_t - b(s_t)\right) \nabla_\theta \log \pi_\theta(a_t|s_t)$$

$$\mathbb{E}\big[\nabla \log \pi \cdot b(s)\big] = b(s)\, \mathbb{E}[\nabla \log \pi] = 0$$

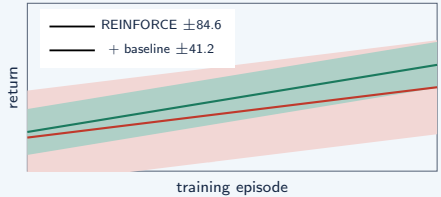
BEST BASELINE $\Rightarrow$ THE ADVANTAGE

$$b(s) = V^\pi(s), \qquad A(s,a) = Q(s,a) - V(s)$$

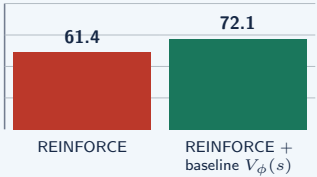
$A > 0$: better than average $\Rightarrow$ increase probability; $A < 0$: worse $\Rightarrow$ decrease;

$A = 0$: no update. The baseline must not depend on a.

Baselines in Practice — Variance Halved With No Bias



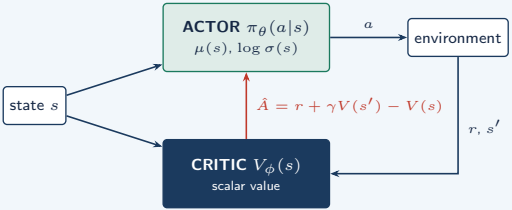
Schematic learning curves; the band widths are the reported return standard deviations.



Waypoint success (%): +10.7 points from the baseline alone.

Bias–variance trade-off: REINFORCE is unbiased but high-variance; a $V(s)$ baseline cuts variance at NO bias. Actor–critic (next) cuts variance further by bootstrapping — at the cost of some bias from the value estimate. PPO controls this trade-off carefully.

Actor–Critic — Two Networks, One Learning Loop



LOSSES

$$L_{\text{actor}} = -\mathbb{E}\big[\hat{A}(s,a)\log \pi_\theta(a|s)\big]$$

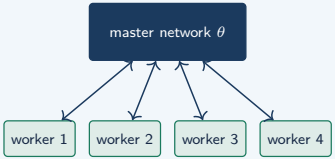
$$L_{\text{critic}} = \mathbb{E}\big[(G_t - V_\phi(s))^2\big]$$

$$L(\theta,\phi) = L_{\text{actor}} + c_1 L_{\text{critic}} - c_2 H(\pi_\theta)$$

- **Actor:** discrete → softmax over actions; continuous → Gaussian $\mu(s)$, $\log \sigma(s)$
- **Critic:** supervised regression of returns
- **TD advantage** $\hat{A} = r + \gamma V_\phi(s') - V_\phi(s)$: the one-step TD error — much lower variance than Monte-Carlo G_t, slight bias from V_ϕ errors
- **Entropy bonus** $H(\pi_\theta)$ keeps the policy exploring

Military analogy: the actor is the platoon commander choosing moves; the critic is the staff officer estimating how good each situation is, so every move can be judged immediately instead of waiting for the battle's outcome.

A2C and A3C — Parallel Workers for Lower Variance



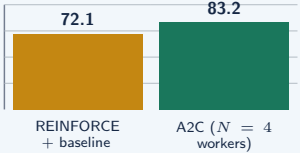
A2C: all workers update together (synchronous)

A2C — synchronous

N parallel environment workers collect experience simultaneously; the master updates from ALL together: more diverse experience, better variance reduction (N samples per update), no staleness. Drone case: $N = 4$ (one per drone).

A3C — asynchronous (Mnih 2016)

Each worker reads θ, runs episodes and pushes gradients on its own schedule. Very efficient on CPUs (no GPU needed); diverse exploration. Weakness: **stale gradients** (workers use an old θ), unstable at high parallelism.



Waypoint success (%): +11.1 points; return $194.8 \pm \mathbf{28.4}$ — variance reduced again.

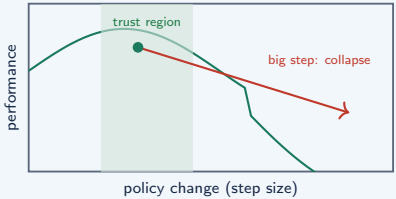
Proximal Policy Optimisation — Why Updates Must Stay Close

Problem with vanilla policy gradient: a large step can change the policy dramatically and performance can **collapse** (e.g. π suddenly starts crashing all drones); recovery takes thousands of episodes.

Trust-region idea (TRPO, 2015)

Maximise the objective subject to $\mathrm{KL}(\pi_{\text{old}} \| \pi_{\text{new}}) \leq \delta$ — a proven monotonic-improvement guarantee, but it needs second-order (natural-gradient/Hessian) computation: prohibitive for large networks.

PPO (Schulman 2017): replace the KL constraint with a clipped objective — first-order SGD/Adam only, 3–10× cheaper than TRPO with similar performance. Today's gold standard.



Schematic: the gradient is only trustworthy near the current policy.

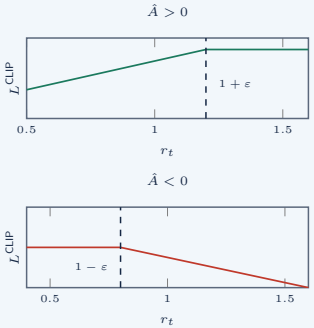
PPO — The Clipped Surrogate Objective

RATIO AND CLIPPED OBJECTIVE

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

$$L^{\text{CLIP}} = \mathbb{E}_t\Big[\min\big(r_t\hat{A}_t,\ \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t\big)\Big]$$

- $r_t = 1$: unchanged; > 1: action more likely; < 1: less likely; typical $\varepsilon = 0.2$
- $\hat{A} > 0$ (good action): gain saturates at $r = 1.2$ — at most 20% more likely per update
- $\hat{A} < 0$ (bad action): saturates at $r = 0.8$ — at most 20% less likely
- No action's probability can move more than $\pm 20\%$ per update $\Rightarrow$ no catastrophic collapse



Flat regions = zero gradient: no incentive to push the policy further.

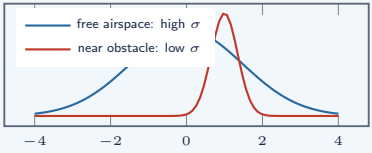
GAUSSIAN POLICY AND REPARAMETERISATION

$$\pi_\theta(a|s) = \mathcal{N}\big(\mu_\theta(s), \mathrm{diag}(\sigma_\theta(s)^2)\big)$$

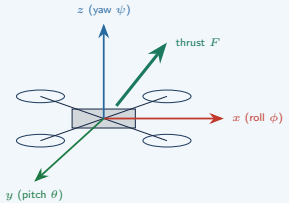
$$a = \mu_\theta(s) + \sigma_\theta(s) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

$$\log \pi_\theta(a|s) = -\tfrac{1}{2} \sum_i \left[\tfrac{(a_i - \mu_i)^2}{\sigma_i^2} + \log \sigma_i^2 + \log 2\pi \right]$$

- The network outputs $\mu_\theta(s)$ (nominal action) and $\log \sigma_\theta(s)$ (learned exploration; more stable than σ)
- Reparameterisation lets gradients flow through the sample; the log-probability is the term differentiated in the policy gradient



Bounded, state-dependent exploration: high σ in open high-altitude airspace (explore!), low σ near obstacles (precise, focused actions). σ is learned from the reward signal automatically.



Six degrees of freedom

3 translational (x, y, z) + 3 rotational (roll, pitch, yaw).

State per drone (18) and action (3)

Position $[x, y, z]$; velocity $[v_x, v_y, v_z]$; orientation [roll, pitch, yaw]; angular rate $[p, q, r]$; fuel $F \in [0, 1]$; waypoint offset $[\Delta x, \Delta y, \Delta z] = 16$, + 2 safety flags = 18. Action: thrust $[F_x, F_y, F_z]$ (N); rotor-speed differential for yaw, simplified to 3-D.

SIMPLIFIED EULER DYNAMICS ($m = 1.2$ KG, $\Delta t = 0.1$ S)

$$v_{t+1} = v_t + \left(F_t/m - g\hat{z}\right)\Delta t, \qquad x_{t+1} = x_t + v_{t+1}\Delta t$$

Swarm coordination reward: bonus for keeping inter-drone spacing ≤ 50 m; penalty for spacing < 10 m (collision risk).

Soft Actor–Critic — Optimise Reward AND Entropy

MAXIMUM-ENTROPY OBJECTIVE

$$J(\pi) = \mathbb{E}\Big[\sum_t \gamma^t \big(r(s_t, a_t) + \alpha H(\pi(\cdot|s_t))\big)\Big]$$

$$H(\pi(\cdot|s)) = -\mathbb{E}\big[\log \pi(a|s)\big]$$

- ◯1 automatic exploration: high-entropy policies try more actions
- ◯2 robustness: stochastic policies are harder for adversaries to predict
- ◯3 multi-modal problems: equally good options all keep probability
- ◯4 temperature α learned automatically (target entropy $= -\dim A$)

SAC is off-policy (replay buffer, reuses data: very sample-efficient for continuous control); **PPO is on-policy** (only current-policy data: more samples, very stable).

SAC ($\alpha = 0.2$, auto): waypoint success **96.2%** (best); return 218.4 ± 18.7 (lowest variance); 1,500 episodes to 70% success; collision rate **1.8%** (lowest).



emergent spread-then-converge formation

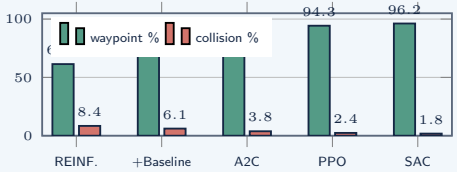
Emergent formation flying

With only the swarm-spacing reward + entropy bonus, drones spontaneously spread out to cover airspace, then converged on each waypoint. Observed spacing **32.4 m** — between the 10 m collision penalty and 50 m bonus boundary. Nobody programmed "keep formation".

Full Results — REINFORCE → PPO → SAC on the 4-Drone Swarm

Algorithm	Type	Waypoint	Collision	Return sd	Eps to 70%	Sample eff.	Key feature
REINFORCE	MC PG	61.4%	8.4%	±84.6	6,200*	very low	simple; unbiased
REINFORCE + $V(s)$	MC PG	72.1%	6.1%	±41.2	3,800	low	variance halved
A2C (N=4)	actor–critic	83.2%	3.8%	±28.4	2,000	medium	parallel workers
PPO (ε=0.2, 10 epochs)	on-policy AC	94.3%	2.4%	±19.8	800	high	clipped, stable
SAC (α=0.2, auto)	off-policy	**96.2%**	**1.8%**	**±18.7**	1,500	very high	entropy ⇒ robust

* The PPTX lists 6,200 for REINFORCE in this column but elsewhere reports 6,200 episodes to 50% success; REINFORCE never reached 70% (61.4% final).



SAC wins on every metric (96.2% success, 1.8% collision, best robustness). **PPO** is simpler to tune and nearly as good (94.3%, stable). Deploy PPO for simpler tasks under training-compute limits; SAC for complex continuous control needing robustness.

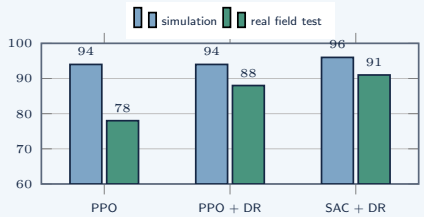
Use policy gradients (PPO/SAC) when

the action space is continuous (thrust, heading, speed); a stochastic policy is needed (exploration, robustness); discrete spaces are huge (> 1000 actions); entropy-based exploration helps.

Use DQN (value-based) when

the discrete action space is small (wargame: 7 actions); off-policy data reuse is critical; interpretable Q-values are needed for an advisory role (Lecture 10).

Sim-to-real: success in simulation vs field (%)



Wind, sensor noise and GPS drift caused a 16-point drop. **Domain randomisation (DR):** random wind ± 5 m/s and sensor noise ± 2 m in training closes most of the gap.

Steering, flocking and learning to move are old problems in every science

AERONAUTICS / CONTROL
PID control (Minorsky, 1922, for ship steering) still flies most drones. Learned policies are often layered on top of, or tune, classical controllers.
Defence: keep a verified PID inner loop; let RL set waypoints and gains.

BIOLOGY
Starling flocks coordinate through each bird tracking about 6–7 nearest neighbours (Ballerini et al., 2008); Reynolds' "boids" (1987) reproduce flocking with three local rules.
Defence: the SAC swarm's emergent spacing is the same phenomenon.

PSYCHOLOGY / MOTOR CONTROL
Fitts' law (1954): movements to small, distant targets take longer — speed and precision trade off; the brain is noisy and plans for it.
Defence: like the learned σ — precise near obstacles, loose in open air.

ENGINEERING / ROBOTICS
OpenAI's Dactyl robot hand (2018–19) learned dexterous manipulation with PPO in simulation using heavy domain randomisation, then transferred to real hardware.
Defence: the same recipe closes the drone sim-to-real gap.

STATISTICS / SIMULATION
The likelihood-ratio (score-function) gradient estimator was developed for optimising stochastic simulations (Glynn, 1990) — REINFORCE is the same estimator.
Wargaming: tuning simulation parameters by gradient.

DEFENCE & WARGAMING
Continuous-control agents can red-team air-defence layouts in simulation by flying thousands of varied ingress paths.
Caution: any armed autonomous behaviour needs meaningful human control and legal review (Lecture 12).

Common thread: good continuous control combines a stable physical model with learned, state-dependent adaptation — and is validated outside the simulator.

Lecture 11 Summary — From REINFORCE to SAC for Continuous Drone Control

POLICY GRADIENT THEOREM
$\nabla J = \mathbb{E}[\nabla \log \pi_\theta(a|s) G_t]$; direct policy optimisation; no argmax; continuous or discrete.
Foundation of all PG methods

REINFORCE
Monte-Carlo returns over complete episodes; unbiased but high variance ($\pm$84.6); 61% waypoint success.
Baseline to beat

ACTOR–CRITIC & A2C
Critic $V_\phi(s)$ cuts variance via TD advantage; A2C with 4 workers: 83%, $\pm$28.4.
Best bias–variance balance

PPO
Clipped ratio $r_t \in [1-\varepsilon, 1+\varepsilon]$ prevents collapse; 94.3% on drones; stable.
Go-to for most RL applications

GAUSSIAN POLICY
$\pi = \mathcal{N}(\mu_\theta, \sigma_\theta^2)$; reparameterised $a = \mu + \sigma\epsilon$; 12-D thrust; σ learned.
Essential for physical control

SAC
Maximum entropy: reward $+\alpha H(\pi)$; off-policy; 96.2%; most robust; emergent formation.
Best sample efficiency + robustness

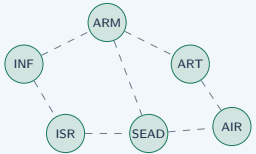
From one agent to a team

- MARL frameworks: cooperative vs competitive settings
- CTDE: centralised training, decentralised execution
- MAPPO, QMIX, MADDPG
- Credit assignment and emergent communication
- Autonomous combat platforms; ethics and human control

Assignment 11 is released with this lecture pack.

Lecture 12 case study

6-unit combined-arms MARL: air–ground coordination and the discovery of emergent tactics.



six cooperating agents, one shared mission